

Checkers – Java

Rapport final

RANDRIAMIHAJA Latiana
RAMANANAHERISOA Fy-Tommy
HOAREAU Faniry
ATWI Daniel
RAZAFINDRAKOTO Sarah

6 avril 2026

Introduction

1 Contexte et objectifs du projet

Le projet a pour objectif de concevoir et développer un logiciel permettant de jouer au jeu de dames conformément aux règles officielles. Le programme doit permettre à deux joueurs de s'affronter sur un damier, soit en mode local, soit à distance via un réseau.

Le logiciel garantit le respect strict des règles du jeu tout en proposant une expérience utilisateur claire, robuste et extensible. Il s'inscrit dans une démarche de développement logiciel complète, intégrant la qualité du code, la documentation, les tests et les performances.

1.1 Variantes supportées

Le jeu de dames se décline en plusieurs variantes internationales, que le programme supporte via une option de configuration du plateau. Le style anglo-saxon, appelé Draughts, se joue sur un plateau de 8×8 , chaque joueur disposant de 12 pions. Le jeu de dames international adopte quant à lui un plateau de 10×10 , avec 20 pions par joueur. Enfin, les dames canadiennes étendent encore la surface de jeu avec un plateau de 12×12 , portant le nombre de pions à 30 par joueur.

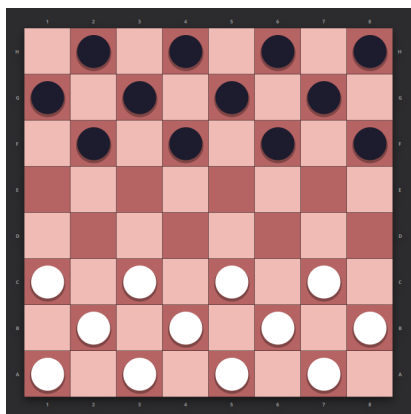


FIGURE 1 – Plateau 8×8

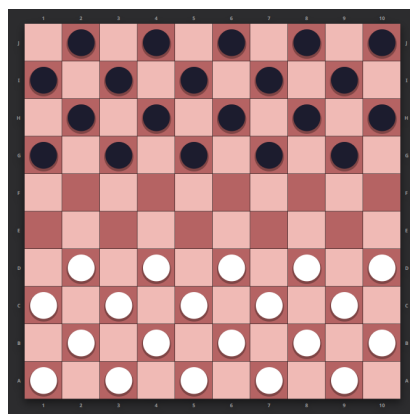


FIGURE 2 – Plateau 10×10

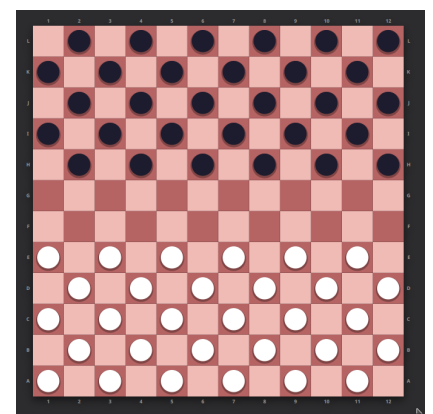


FIGURE 3 – Plateau 12×12

1.2 Règles du jeu

1.2.1 Mise en place et déroulement

Le damier est orienté avec une case foncée en bas à gauche. Les pions se placent exclusivement sur les cases foncées des quatre premières rangées de chaque camp. Les joueurs jouent chacun à leur tour ; les blancs commencent toujours.

1.2.2 Déplacements

Un pion se déplace d'une case vers l'avant en diagonale. Lorsqu'il atteint la dernière rangée adverse et s'y arrête, il est promu en dame, ce qui se matérialise visuellement par l'ajout d'un point jaune. La dame bénéficie alors d'une mobilité bien supérieure : elle peut parcourir librement une diagonale entière, d'autant de cases qu'elle le souhaite, aussi bien vers l'avant que vers l'arrière.

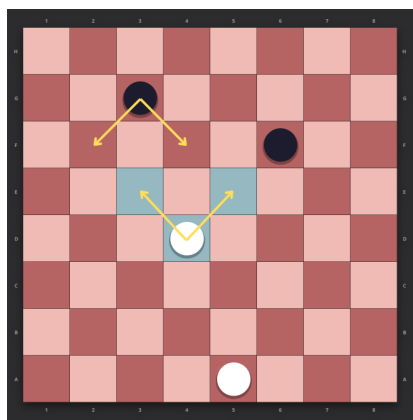


FIGURE 4 – Déplacement d'un pion

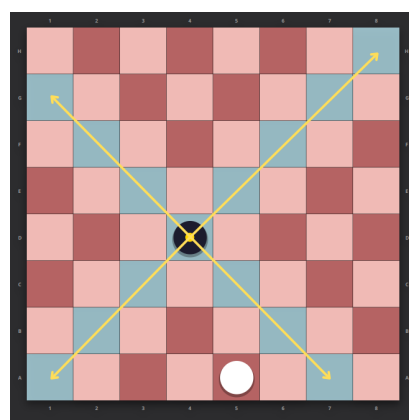


FIGURE 5 – Déplacement d'une dame

1.2.3 La prise par un pion

Un pion capture un pion adverse en sautant par-dessus lui pour se poser sur la case vide située immédiatement derrière. La prise peut s'effectuer vers l'avant comme vers l'arrière.

La prise est obligatoire. De plus, si après une capture le pion se retrouve à nouveau en position de prise, il doit continuer jusqu'à épuisement des captures possibles (rafle). Les pions capturés sont retirés du plateau uniquement à l'issue complète de la rafle, et non au fur et à mesure.

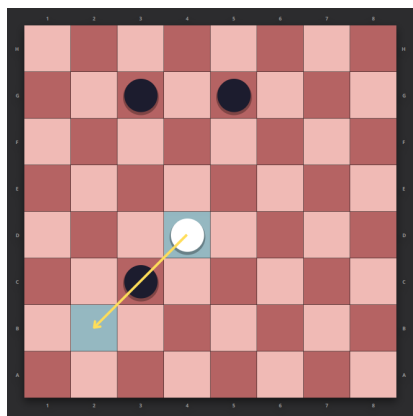


FIGURE 6 – Prise d'un pion

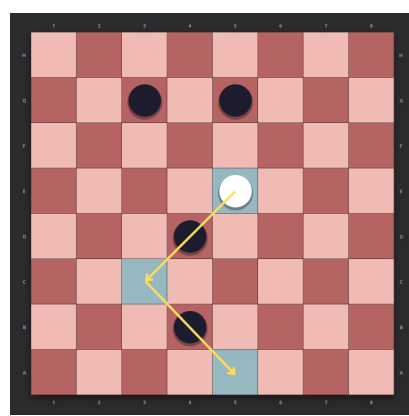


FIGURE 7 – Rafle

1.2.4 La prise majoritaire

Lorsque plusieurs séquences de capture sont possibles, le joueur est tenu de choisir celle qui capture le plus grand nombre de pièces, quel que soit leur rang. Ainsi, une séquence permettant de prendre deux pions est prioritaire sur la prise d'une dame isolée.

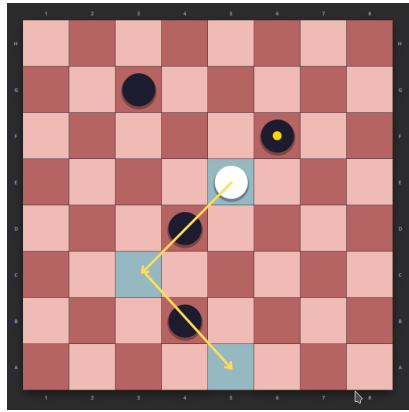


FIGURE 8 – Prise majoritaire : 2 pions vs 1 dame

1.2.5 La prise par la dame

Grâce à son amplitude de mouvement, la dame dispose de capacités de capture étendues. Elle doit prendre tout pion situé sur sa diagonale dès lors qu'une case libre est disponible immédiatement après. On ne peut passer qu'une seule fois sur un même pion, mais on peut traverser deux fois la même case.

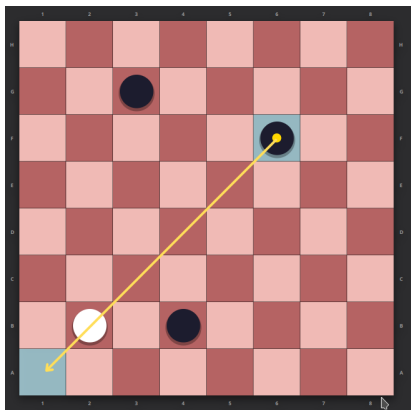


FIGURE 9 – Prise par la dame

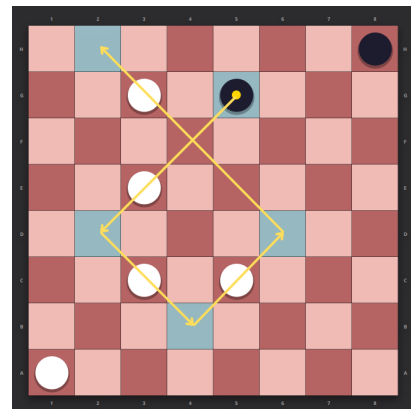


FIGURE 10 – Prise par la dame avec passage sur une même case

1.3 Conditions de fin de partie

Une partie prend fin dans l'un des cas suivants :

Victoire par capture. Un joueur remporte la partie en capturant l'intégralité des pièces adverses.

Victoire par blocage. Si un joueur ne peut effectuer aucun déplacement alors qu'il lui reste des pièces, il perd immédiatement la partie.

Nulle par accord de position. La partie est déclarée nulle si la même position se reproduit trois fois au cours de la partie (règle de la répétition triple).

Nulle par absence de progrès. Si aucune capture ni aucun mouvement de pion n'est effectué pendant 25 tours consécutifs (soit 50 demi-coups), la partie est déclarée nulle.

Nulle en fin de partie. Lorsqu'un joueur ne dispose plus que d'une seule pièce, si aucune issue décisive n'est atteinte en 16 tours (soit 32 demi-coups), la partie est déclarée nulle faute de matériel suffisant.

Nulle par blocage mutuel. Dans le cas exceptionnel où aucun des deux joueurs ne peut effectuer de mouvement, la partie est également déclarée nulle.

1.4 Versions existantes

Dans le cadre de notre phase d'analyse, nous avons étudié deux projets open-source de jeu de dames afin d'identifier des approches architecturales et algorithmiques pertinentes pour notre développement.

AwsDame

AwsDame [2] est une plateforme de jeu de dames multijoueur en temps réel développée dans un environnement JavaScript/Node.js. Bien que la pile technologique diffère de la nôtre (Java), son architecture réseau orientée événements offre des points de comparaison instructifs.

En particulier, ce projet adopte une logique de *matchmaking automatique* entre joueurs connectés, une idée que nous avons reprise dans notre propre serveur TCP. Notre classe `GameServer` implémente en effet une méthode `tryAutoStart()` qui détecte automatiquement la disponibilité de deux joueurs inactifs et déclenche une session sans intervention manuelle :

```
// GameServer.java -- démarrage automatique d'une partie
List<PlayerSession> idlePlayers = registry.getAllPlayers().stream()
    .filter(PlayerSession::isIdle)
    .collect(Collectors.toList());

if (idlePlayers.size() >= 2) {
    List<PlayerSession> pair = idlePlayers.subList(0, 2);
    GameSession session = registry.createSession(pair, sessionController);
    // notification GAME_START envoyée aux deux joueurs
}
```

Contrairement à **AwsDame** qui s'appuie sur WebSockets, notre solution repose sur des sockets TCP bruts avec un protocole textuel simple (REGISTER, MOVE, GAME_OVER, etc.), ce qui facilite les tests en ligne de commande et l'intégration avec un client Java natif.

Raven Checkers

Raven Checkers [1] est un programme de dames open-source développé en Python, conçu à des fins pédagogiques pour l'étude des tactiques de jeu et des algorithmes d'intelligence artificielle. Il couple la fonction d'évaluation de *Simple Checkers* aux algorithmes de recherche du projet AIMA [3].

L'analyse de ce projet a principalement influencé notre implémentation des algorithmes d'IA. **Raven Checkers** propose un MinMax classique sans gestion du temps, ce qui le rend peu adapté à des contraintes de jeu réel. Notre implémentation y répond de trois façons :

1. **Approfondissement itératif.** Plutôt que de fixer une profondeur statique, nos classes `MinMax` et `MinMaxAlphaBeta` explorent l'arbre niveau par niveau, de la profondeur 1 jusqu'à `maxDepth`, et conservent uniquement le résultat du dernier niveau entièrement exploré. Si le budget temps expire en cours de niveau, l'exploration partielle est abandonnée via une exception interne `TimeExceededException`, garantissant qu'aucun résultat incomplet n'est retenu.
2. **Élagage alpha-bêta.** La classe `MinMaxAlphaBeta` maintient deux bornes α (meilleur score garanti pour le maximisant) et β (meilleur score garanti pour le minimisant). Dès que $\beta \leq \alpha$, les branches restantes sont élaguées, réduisant significativement le nombre de nœuds évalués par rapport au MinMax pur.
3. **MCTS avec mode de sélection configurable.** Au-delà du MinMax, notre classe `Mcts` implémente la méthode de Monte Carlo Tree Search, avec un mode de sélection paramétrable (UCT ou ML) défini par l'énumération `SelectionMode`. Cette flexibilité est absente de **Raven Checkers**, qui ne propose qu'un seul algorithme de recherche.

La fabrique `Ai.buildAi(Configuration)` centralise le choix de l'algorithme à l'exécution :

```
// Ai.java -- sélection de l'algorithme via la configuration
switch (aiMode.toLowerCase()) {
    case "minimax" -> return new MinMax(depth, timeMs);
    case "alphabeta" -> return new MinMaxAlphaBeta(depth, timeMs);
    case "mcts" -> {
        Mcts mcts = new Mcts(depth, timeMs);
        mcts.setSelectionMode(cfg.getSelectionMode());
        return mcts;
    }
}
```

Bilan comparatif

Le tableau 1 synthétise les principales différences entre les deux projets analysés et notre propre implémentation.

L'étude de ces deux projets nous a permis de valider nos choix de conception et d'identifier les lacunes à combler : gestion du temps dans la recherche adversariale, richesse algorithmique de l'IA, et protocole réseau adapté à un client Java interactif.

TABLE 1 – Comparaison des projets existants et de notre implémentation

Critère	AwsDame	Raven Checkers	Notre projet
Langage	JavaScript	Python	Java
Transport réseau	WebSocket	—	TCP (sockets)
Matchmaking automatique	Oui	—	Oui
Suivi des scores	—	—	Oui (W/L/D)
MinMax	—	Oui	Oui
Élagage alpha-bêta	—	Non	Oui
Approfondissement itératif	—	Non	Oui
MCTS	—	Non	Oui
Contrainte de temps pour l'IA	—	Non	Oui (configurable)

2 Algorithme et Structure de données

2.1 Bitboard

Nous utilisons des bitboards pour représenter l'état du plateau. Cette approche permet de représenter l'état du plateau sous forme d'entiers primitifs et de calculer les coups de manière globale via des opérations bit à bit, offrant des performances bien supérieures aux matrices 2D classiques.

2.1.1 Représentation et Numérotation des cases

Sur les 100 cases d'un plateau standard (10×10), seules les 50 cases noires sont utilisées. Nous utilisons la **notation de Manoury** pour numéroter ces cases actives de 0 à 49, de bas en haut et de gauche à droite.

L'état complet du jeu est encodé sur quelques entiers de 64 bits (`long` en Java), où chaque bit correspond à une case active (1 si la case est occupée, 0 si elle est vide). Cela nous évite d'écrire des formules mathématiques complexes pour définir l'état du plateau : tout repose sur l'indexation binaire.

	.	45	.	46	.	47	.	48	.	49
40	.	41	.	42	.	43	.	44	.	
	.	35	.	36	.	37	.	38	.	39
30	.	31	.	32	.	33	.	34	.	
	.	25	.	26	.	27	.	28	.	29
20	.	21	.	22	.	23	.	24	.	
	.	15	.	16	.	17	.	18	.	19
10	.	11	.	12	.	13	.	14	.	
	.	5	.	6	.	7	.	8	.	9
0	.	1	.	2	.	3	.	4	.	

2.1.2 Gain de mémoire chiffré

L'intérêt premier du bitboard est la compression des données. Comparons cette approche avec une structure de données "classique" comme un tableau à deux dimensions en Java (`int[10][10]`). Un tableau 2D classique nécessite 100 cases, chacune stockant un entier de 32 bits, soit un total de 3200 bits pour représenter l'état du plateau. En revanche, avec les bitboards, nous représentons l'ensemble du plateau avec seulement 4 variables de type `long` (64 bits chacune) : une pour les pions blancs, une pour les pions noirs, un pour les dames blanches et un pour les dames noires. Cela représente seulement 256 bits au total.

Le gain en mémoire vive est d'un facteur 12, ce qui accélère drastiquement les copies de plateaux lors de la recherche de l'IA.

2.1.3 Application des opérations binaires

L'avantage majeur du bitboard n'est pas seulement le stockage, mais la manipulation du plateau via le processeur. Au lieu de parcourir le plateau case par case, nous utilisons :

- **ET logique** ($\wedge$) : Pour détecter les collisions (ex : croiser un masque de déplacement avec le bitboard des pièces adverses).
- **OU logique** ($\vee$) : Pour fusionner les bitboards (ex : Blancs $\vee$ Noirs) et obtenir la carte de toutes les cases occupées.

- **NON logique** ($\neg$) : Pour inverser la carte d'occupation et isoler instantanément toutes les cases vides du damier.
- **Décalage binaire** ($\ll$, $\gg$) : Pour traduire toutes les pièces d'un joueur dans une direction donnée en une seule instruction d'horloge.

2.2 Algorithmes de manipulation

L'intérêt de la section précédente prend tout son sens ici : la génération des coups légaux repose sur la translation temporelle des bits.

TABLE 2 – Comparaison des décalages binaires selon la taille du plateau

Taille Plateau	8x8 (32 bits)		10x10 (50 bits)		12x12 (72 bits)	
Transition Ligne	P → I	I → P	P → I	I → P	P → I	I → P
Haut-Gauche	+3	+4	+4	+5	+5	+6
Haut-Droite	+4	+5	+5	+6	+6	+7
Bas-Gauche	-5	-4	-6	-5	-7	-6
Bas-Droite	-4	-3	-5	-4	-6	-5

2.2.1 Déplacement et prise par un pion

Lorsqu'un pion se déplace, on décale l'ensemble de son bitboard d'une direction donnée. Si le bit "atterrit" sur une position valide (une case vide, vérifiée via une opération $\wedge$ sur le bitboard inversé $\neg(\text{Occupées})$), le coup est légal.

Exemple concret de prise : Pour qu'une capture soit possible, un pion doit sauter par-dessus un pion adverse et atterrir sur une case vide. Sur un plateau 10×10 , supposons un pion blanc en case 32 (ligne paire). S'il veut capturer un pion noir situé en haut à gauche (case 27, ligne impaire), le décalage pour atteindre le pion noir est de +5. Pour atterrir derrière ce pion, le saut continue vers la case 21 (ligne paire), nécessitant un nouveau décalage de +6.

Le décalage total pour cette prise est donc constant : $5 + 6 = 11$ bits. L'opération binaire simplifiée de cette validation ressemble à cela :

$$\text{CibleSaut} = (\text{PionBlanc} \gg 5) \wedge \text{PionNoir}$$

$$\text{AtterrissageValide} = (\text{CibleSaut} \gg 6) \wedge \neg(\text{Occupées})$$

2.2.2 Déplacement et prise par une dame

Contrairement au pion, la dame peut parcourir un nombre illimité de cases vides sur une diagonale. D'un point de vue binaire, un simple décalage ne suffit pas : il faut propager le bit sur toute la diagonale jusqu'à rencontrer un obstacle. C'est la technique du *sliding piece generation*.

Voici le pseudo-code illustrant la génération des coups possibles pour une dame :

```
Fonction GenererCoupsDame(bitboard_dame, direction):
    coups_possibles = 0
    // Premier decalage dans la direction souhaitee
    rayons = decaler(bitboard_dame, direction)

    // Tant qu'on se deplace sur des cases vides
    Tant que rayons != 0:
        // On valide les cases vides rencontrees comme coups possibles
        coups_possibles = coups_possibles OU (rayons ET cases_vides)

        // On continue la propagation uniquement depuis les cases vides
        // La propagation s'arrete si on touche une piece ou le bord
        rayons = decaler(rayons ET cases_vides, direction)

    Retourner coups_possibles
```

3 Architecture du projet

3.1 Le Moteur de Jeu

L'architecture de notre application repose sur le patron Modèle-Vue-Contrôleur (MVC). Cette section détaille la conception de notre moteur de jeu (qui incarne le Modèle) et la façon dont il interagit avec le reste du programme pour assurer le bon déroulement d'une partie.

3.1.1 Rôle et responsabilités

Le moteur de jeu, regroupé dans le package `model`, a pour unique mission de simuler et de faire tourner la partie de dames. Il se concentre purement sur la logique mathématique et les règles officielles : il ne gère absolument pas les clics de souris, la saisie clavier ou l’affichage graphique.

Ses responsabilités sont réparties entre deux classes majeures. D’un côté, la classe `Board` agit comme le moteur physique du damier : elle manipule les pièces, valide si un coup respecte la géométrie du plateau et applique les règles de prises obligatoires. De l’autre, la classe `GameCheckers` incarne l’arbitre de la partie. C’est elle qui maintient l’état global du système. Elle garde la trace du joueur actif, stocke l’historique des mouvements pour permettre les retours en arrière (undo/redo), et détecte les conditions de fin de partie (victoire, blocage, ou match nul par répétition).

3.1.2 Interface de communication (API)

Pour que le reste de l’application puisse interagir avec ce moteur, `GameCheckers` expose une interface de communication claire basée sur des actions d’entrée et des notifications de sortie.

En entrée, l’interface graphique ou textuelle ne modifie jamais le plateau directement. Elle passe par le `GameController` qui interroge d’abord le moteur via `getPossibleMoves(player)` pour s’assurer qu’une action est légale, puis lui demande de l’exécuter avec la méthode `applyMove(from, to)`.

En sortie, le moteur dicte le rythme grâce au patron de conception Observateur. Au lieu de forcer l’interface graphique à vérifier constamment si quelque chose a changé, le moteur travaille en silence et émet un signal de mise à jour via `notifyObservers()` uniquement lorsqu’un mouvement valide vient de modifier l’état du jeu.

3.1.3 Choix techniques et boucle de contrôle

D’un point de vue technique, la représentation du plateau ne repose pas sur un simple tableau à deux dimensions, mais sur des *bitboards* (comme détaillé dans la section algorithmique). Ce choix garantit un accès et une manipulation en temps $O(1)$ via des opérations binaires.

Concernant la dynamique, le jeu s’exécute au tour par tour, géré par une machine à états simple définie par l’énumération `State` (`IN_GAME`, `PAUSE`, `FINISHED`). L’évolution du système lors de l’application d’un coup suit toujours la même séquence logique stricte :

```
// Extrait simplifié de GameCheckers.java : évolution de l'état du système
public void applyMove(String from, String to, boolean isManoury) {
    // 1. Délégation de la physique au plateau
    board.applyMove(move);

    // 2. Mise à jour de l'état global (historique et tour de jeu)
    managerUndoRedo.registerMove(currentColor, move);
    this.isWhiteTurn = !this.isWhiteTurn;

    // 3. Signal de fin d'opération à l'interface
    notifyObservers();
}
```

Une fois le coup appliqué et la vue rafraîchie, le `GameController` vérifie si la partie est terminée. Si le jeu continue, il oriente le flux vers une attente de saisie utilisateur, ou lance immédiatement la réflexion de l’algorithme si le joueur suivant est une intelligence artificielle.

3.1.4 Indépendance du moteur

3.2 Module affichage

Le module d’affichage constitue la couche Vue de notre architecture. Son objectif est de transformer l’état métier fourni par le moteur en informations compréhensibles par l’utilisateur, puis de remonter les intentions de jeu vers le contrôleur.

3.2.1 Responsabilités du module

Ce module ne contient aucune règle de dames. Il se limite à trois responsabilités :

- **Rendu de l’état courant** : représentation du plateau, des pièces, du joueur actif et des messages de statut.
- **Capture des actions utilisateur** : sélection d’une case, validation d’un coup, demandes de pause / sauvegarde / chargement.
- **Retour d’information** : affichage des coups illégaux, de la fin de partie et des événements de synchronisation réseau.

3.2.2 Affichage en mode CLI

Le mode CLI (Command Line Interface) fournit une interface textuelle légère, utile pour les tests rapides, le débogage et l'exécution sur des environnements sans interface graphique. Dans ce mode, la Vue sérialise l'état du damier en texte (coordonnées, pièces, tour courant), puis lit des commandes utilisateur de type `MOVE`, `UNDO`, `SAVE` ou `LOAD`.

D'un point de vue architectural, la CLI reste alignée sur MVC : elle ne touche jamais au `Board` directement. Chaque commande est traduite en intention métier et transmise au `GameController`, qui valide le coup et orchestre l'appel au moteur. Ce fonctionnement garantit que les mêmes règles sont appliquées, quel que soit le canal d'entrée (texte ou interface graphique).

Le mode CLI joue également un rôle important pour les tests automatisés du projet : son comportement déterministe facilite l'écriture de scénarios reproductibles (entrées textuelles connues, sorties attendues), notamment pour vérifier les cas limites des règles, le routage des commandes et la robustesse des opérations de sauvegarde/chargement.

3.2.3 Affichage en mode GUI

Le mode GUI (Graphical User Interface) vise une expérience utilisateur interactive et visuelle. Il représente le plateau, les pièces et les états de partie sous forme de composants graphiques, et gère des événements riches (clic sur case, sélection d'une pièce, surbrillance des coups possibles, messages contextuels).

Comme pour la CLI, la GUI n'intègre pas la logique des règles : elle agit comme un adaptateur événementiel. Lorsqu'un utilisateur clique sur une case de départ puis une case d'arrivée, l'action est convertie en requête métier vers le `GameController`. Après traitement par `GameCheckers`, la mise à jour de l'affichage est déclenchée par le mécanisme d'observation (`notifyObservers()`).

Cette séparation permet de conserver une GUI réactive tout en évitant les incohérences fonctionnelles. Elle facilite aussi l'évolution de l'interface (thèmes, animations, ergonomie) sans impact sur le moteur, puisque le contrat entre Vue et contrôleur reste inchangé.

3.2.4 Interaction avec le moteur MVC

Le flux d'interaction suit une chaîne stricte :

1. L'utilisateur agit via l'affichage (clic, commande, sélection).
2. Le module Vue transmet l'intention au `GameController`.
3. Le `GameController` valide et délègue l'opération au moteur (`GameCheckers/Board`).
4. Après mise à jour, le moteur notifie ses observateurs.
5. Le module d'affichage reçoit la notification et rafraîchit l'interface.

Ce schéma garantit que la Vue ne modifie jamais directement l'état du jeu : elle reste consommatrice de l'état produit par le modèle.

3.2.5 Dépendances et extensibilité

Les dépendances sont orientées de la Vue vers le contrôleur, jamais l'inverse. Le module d'affichage dépend d'interfaces publiques stables (état de partie, coups légaux, notifications), ce qui permet :

- d'ajouter un nouveau type d'interface sans toucher au moteur ;
- d'exécuter des tests du moteur sans initialiser d'interface graphique ;
- de conserver une cohérence fonctionnelle entre mode local, IA et réseau.

4 Explication approfondie : Monte Carlo Tree Search (MCTS)

4.1 Logique théorique de l'algorithme

Contrairement à l'algorithme Minimax qui tente d'évaluer exhaustivement toutes les branches jusqu'à une profondeur donnée, le MCTS repose sur des simulations statistiques probabilistes. La recherche s'effectue en répétant une boucle de quatre phases fondamentales jusqu'à épuisement du temps de réflexion imparti :

4.1.1 Sélection

À chaque itération, l'algorithme part de la racine de l'arbre (l'état actuel du damier) et descend vers les feuilles. Pour choisir quel chemin emprunter, il utilise la formule UCT (Upper Confidence Bound applied to Trees) :

$$UCT_j = \frac{w_j}{n_j} + C \sqrt{\frac{\ln n}{n_j}}$$

Où :

- w_j représente le nombre de victoires accumulées par les simulations passant par le nœud enfant j .

- n_j est le nombre total de simulations ayant traversé ce nœud j .
- n est le nombre total de visites du nœud parent.
- C est la constante d'exploration.

Concrètement : Cette formule résout le dilemme "Exploration vs Exploitation". Le terme de gauche favorise les coups ayant un fort taux de victoire (exploitation), tandis que le terme de droite grandit pour les coups peu testés (exploration). Si un nœud enfant n'a jamais été visité, sa valeur est considérée comme infinie pour forcer l'algorithme à tester chaque coup légal au moins une fois.

4.1.2 Expansion

Une fois arrivé sur un nœud dont tous les coups possibles n'ont pas encore été explorés, l'algorithme choisit un de ces mouvements non testés, crée un nouveau nœud dans l'arbre pour représenter ce nouvel état, et passe à la phase suivante.

4.1.3 Simulation

Depuis ce nœud nouvellement créé, l'algorithme lance une partie virtuelle. Il joue des coups totalement aléatoires pour les deux joueurs jusqu'à atteindre un état final (victoire, défaite ou égalité).

4.1.4 Rétropropagation

Une fois la partie simulée terminée, le résultat (victoire, défaite ou nul) est remonté le long de l'arbre en suivant le chemin exact pris lors de la phase de sélection. Tous les nœuds ancêtres traversés voient leurs statistiques mises à jour : incrémentation de leur compteur de visites, et ajustement de leur score de victoire selon le camp gagnant.

4.2 Implémentation et optimisations dans notre projet

Notre classe `Mcts` gère cette recherche de manière optimisée en traduisant cette théorie à travers des choix d'architecture précis.

4.2.1 Traduction des phases en code

La boucle principale est gérée par la méthode `calculateBestMove()`, qui s'exécute tant que la limite de temps (`isTimeExceeded`) n'est pas atteinte.

Lors de la phase de **sélection**, la formule UCT est appliquée par `bestChildByUcb1()`. Si $n_j = 0$, la méthode renvoie `Double.MAX_VALUE` pour assurer l'exploration initiale :

```
double score = child.visits == 0
    ? Double.MAX_VALUE
    : child.winRate() + explorationConstant * Math.sqrt(logVisits / child.visits);
```

Lors de la **simulation** (`simulate()`), pour éviter des parties virtuelles infinies sur des situations bloquées, nous avons introduit une limite stricte de profondeur (`SIMULATION_DEPTH = 60`). Si cette limite est atteinte sans vainqueur, la partie rapporte un score de parité (`DRAW_SCORE = 0.5`), contre `WIN_SCORE = 1.0` pour une victoire.

4.2.2 Optimisation mémoire (ManagerUndoRedo)

L'un des plus grands défis du MCTS est la gestion de la mémoire. Au lieu de cloner l'objet `Board` des milliers de fois pour chaque nœud et simulation (ce qui saturerait la RAM et ralentirait drastiquement l'exécution), nous utilisons une instance unique du plateau couplée à un `ManagerUndoRedo`. Chaque coup testé lors de la sélection, de l'expansion et de la simulation est appliqué physiquement sur le plateau, puis proprement annulé lors de la phase de `backpropagate` :

```
// Rétropropagation : on remonte jusqu'à la racine en annulant les coups joués
while (current != null) {
    current.visits++;
    // Mise à jour des scores...

    // Annulation physique du coup sur le plateau unique
    if (current.move != null) {
        undo.undo(current.player == PlayerColor.WHITE);
    }
    current = current.parent;
}
```

4.2.3 Guidage par Machine Learning

Pipeline d'entraînement du modèle Pour générer les poids (`mlWeights`) et le biais (`mlBias`) utilisés par cette fonction, nous avons implémenté un pipeline d'apprentissage supervisé autonome dans la classe `LogisticRegressionTrain`. L'entraînement s'effectue via l'option en ligne de commande `-tr`. Le processus se divise en trois grandes étapes :

1. **Génération des données par Self-Play** : L'application fait s'affronter deux instances de l'IA (MCTS pur) l'une contre l'autre. Pour éviter les parties infinies, une limite stricte de 100 coups est fixée (`MAX_MOVES`). À chaque demi-coup, un vecteur de 6 caractéristiques (*features*) est extrait du plateau :
 - Nombre de pions blancs et noirs.
 - Nombre de dames blanches et noires.
 - Différentiel de pions (Blancs - Noirs).
 - Différentiel de dames (Blancs - Noirs).Si la partie aboutit à une victoire, tous les vecteurs de cette partie sont étiquetés avec le résultat réel (1.0 pour une victoire des blancs, 0.0 pour les noirs). Les parties nulles sont ignorées pour ne pas bruite le modèle.
2. **Optimisation par descente de gradient** : Une fois les données collectées (par défaut sur 10 parties), la méthode `train()` ajuste les poids itérativement sur 1000 *epochs*. À chaque étape, pour chaque vecteur d'état, le modèle effectue une prédiction. L'erreur par rapport au résultat réel (*label*) est calculée, et les poids sont mis à jour selon un taux d'apprentissage (`learningRate = 0.01`) :

$$W_j = W_j - \alpha \times \text{erreur} \times X_j$$

Où α est le taux d'apprentissage, W_j le poids de la caractéristique j , et X_j la valeur de la caractéristique.

3. **Sauvegarde et chargement** : À l'issue de l'entraînement, les poids finaux et le biais sont sauvegardés dans un fichier texte (`ml_weights.txt`). Ce fichier est ensuite chargé dynamiquement par `Mcts` lors de son instantiation, garantissant une séparation propre entre la phase d'entraînement et la phase d'inférence (jeu).

Bilan de l'approche Bien évidemment, ce mode de sélection par apprentissage automatique n'a pas démontré de gain significatif en termes de performance ou de qualité de jeu par rapport à l'UCT classique. Cela s'explique par la simplicité de notre modèle : la régression logistique est un modèle linéaire, et nos 6 *features* mathématiques (uniquement basées sur le comptage matériel) ne capturent pas les subtilités positionnelles du jeu de dames.

Cependant, cette implémentation robuste démontre l'extensibilité de notre architecture. Le pipeline de génération de données, d'entraînement et d'inférence est pleinement fonctionnel, ce qui ouvre la porte à des améliorations futures plus sophistiquées (comme le remplacement de la régression logistique par un réseau de neurones profond évaluant des motifs complexes) pour renforcer drastiquement l'intelligence de notre agent.

4.2.4 Détermination du meilleur coup final

À l'épuisement du temps de réflexion imparti, l'arbre cesse de croître. Quelle que soit l'heuristique de sélection utilisée durant la recherche (la formule UCT classique ou les prédictions du modèle de Machine Learning), la décision finale ne se base pas sur ces scores d'évaluation.

L'IA fait appel à la méthode `mostVisitedChild()`, qui sélectionne le nœud enfant de la racine possédant le **plus grand nombre de visites**. Ce critère est privilégié car il est statistiquement le plus robuste : une branche très visitée signifie qu'elle a continuellement été considérée comme la plus prometteuse par l'algorithme, et qu'elle a survécu à un grand nombre de simulations face aux meilleures réponses possibles de l'adversaire. Cette séparation stricte des responsabilités rend le moteur de jeu totalement agnostique vis-à-vis de l'environnement extérieur. La classe `GameCheckers` ignore complètement si le joueur tape ses commandes dans une console Linux ou s'il déplace une image de pion sur une fenêtre JavaFX.

Ce choix de conception a été décisif pour notre flux de travail. Il nous a permis de développer, de faire tourner et de tester massivement l'intégralité du moteur de jeu et de l'IA via une interface textuelle simple, avant même de commencer à coder l'interface graphique. Remplacer le terminal par la vue JavaFX n'a nécessité aucune modification des classes du moteur.

5 IA et Complexité

5.1 Hypothèses et notations

Dans cette section, nous analysons la complexité temporelle des méthodes principales des classes IA fournies. La variable n désigne le nombre de cases actives du plateau (typiquement 32, 50 ou 72), b représente le facteur de branchement moyen (nombre de coups légaux par position), et d la profondeur maximale de recherche. Nous distinguons la complexité *propre* d'une méthode de recherche (nombre de nœuds explorés) du coût réel, qui inclut aussi la génération des coups par le moteur via `getWhiteValidMoves()` et `getBlackValidMoves()`.

5.2 Lecture intuitive des classes

La classe `SimpleEvaluator` peut être vue comme une balance très basique : elle compte simplement le "matériel" sur le plateau, c'est-à-dire combien de pions et de dames possède chaque joueur. Si les blancs ont plus de pièces fortes, le score monte ; si les noirs sont avantagés, le score baisse. Cette approche est facile à comprendre, rapide, mais elle ne tient pas compte de la position stratégique des pièces.

La classe `AdvancedEvaluator` reprend cette idée de balance, mais ajoute une intuition naturelle du jeu de dames : un pion qui avance vers la promotion devient plus dangereux. En pratique, cette classe récompense les pions qui progressent vers la dernière rangée. Elle reste simple, mais elle joue déjà plus "intelligemment" qu'un simple comptage de pièces.

La classe `MaxEvaluator` va encore plus loin. En plus du matériel et de l'avancement, elle valorise le contrôle du centre du damier et la mobilité (le nombre de coups légaux disponibles). Intuitivement, un joueur qui contrôle le centre et qui garde beaucoup d'options est souvent en meilleure posture. Cette évaluation est plus riche et souvent plus précise, mais elle demande davantage de calcul.

La classe `MinMax` représente la logique "je joue un coup, puis j'anticipe la meilleure réponse adverse". L'algorithme explore un arbre de possibilités en alternant les deux joueurs : il suppose que l'adversaire joue bien, et il cherche le coup qui reste le plus favorable malgré cela. Cette méthode donne des décisions solides, mais son coût augmente rapidement quand on approfondit la recherche.

La classe `MinMaxAlphaBeta` suit exactement la même philosophie que `MinMax`, mais elle évite de calculer des branches qui ne pourront de toute façon pas changer la décision finale. On peut l'imaginer comme une lecture intelligente d'un arbre où l'on saute les chapitres déjà "perdus d'avance". Le résultat reste de même qualité, mais le temps de calcul peut être fortement réduit.

La classe `IterativeDeepening` applique une stratégie très pratique en contexte réel : elle commence par une recherche peu profonde (rapide), puis recommence avec une profondeur plus grande, et ainsi de suite, jusqu'à épuisement du temps autorisé. L'avantage est qu'on dispose toujours d'un coup jouable, même si la recherche complète la plus profonde n'a pas le temps de se terminer.

5.3 Complexité des fonctions d'évaluation

SimpleEvaluator La méthode `evaluate(Board)` réalise un parcours linéaire de l'ensemble des cases actives, avec uniquement des tests bit-à-bit en temps constant à chaque itération. Sa complexité est donc $O(n)$.

AdvancedEvaluator La méthode `evaluate(Board)` suit le même schéma que `SimpleEvaluator` : un balayage linéaire des cases et un calcul de bonus d'avancement en temps constant par case. La complexité reste donc $O(n)$.

MaxEvaluator La méthode `evaluate(Board)` combine trois contributions : un parcours principal en $O(n)$, un test de centralité `isInCenter(...)` en $O(1)$ par case, puis une évaluation de mobilité dont le coût dépend des appels `getWhiteValidMoves()` et `getBlackValidMoves()`. Sa complexité globale s'écrit donc $O(n + M_w + M_b)$, où M_w et M_b représentent les coûts de génération des coups blancs et noirs. La méthode `isInCenter(int, int, int)` elle-même ne fait que des comparaisons arithmétiques constantes et s'exécute en $O(1)$.

5.4 Complexité des algorithmes de recherche adversariale

MinMax La méthode `calculateBestMove(...)` lance une recherche complète jusqu'à `maxDepth` et hérite donc d'une complexité asymptotique en $O(b^d)$, hors coût exact de génération des coups. La méthode `findBestMoveAtDepth(...)` itère d'abord sur les coups racines puis délègue à l'exploration récursive `minMax / maxMin`, ce qui conduit au même ordre de grandeur. Plus finement, `minMax(...)` et `maxMin(...)` coûtent $O(b^k)$ à profondeur restante k , soit $O(b^d)$ au pire depuis la racine. Enfin, `checkTime(long)` effectue un test booléen constant en $O(1)$.

MinMaxAlphaBeta Les méthodes `calculateBestMove(...)` et `findBestMoveAtDepth(...)` suivent la même logique que `MinMax`, mais ajoutent les bornes α et β pour couper des sous-arbres. Ainsi, `maxValue(...)` et `minValue(...)` restent en $O(b^d)$ dans le pire cas (ordre des coups défavorable), mais peuvent atteindre $O(b^{d/2})$ dans le meilleur cas théorique avec un ordre des coups optimal. La méthode `checkTime(long)` reste en $O(1)$.

IterativeDeepening La méthode `calculateBestMove(...)` enchaîne des recherches successives de profondeur $1, 2, \dots, d$ tant que le budget temps n'est pas épuisé. Son coût total théorique est

$$\sum_{k=1}^d O(b^k) = O(b^d).$$

En pratique, le surcoût par rapport à une unique recherche à profondeur d est modéré, et il est compensé par la robustesse de la stratégie : en cas d'interruption temporelle, le meilleur coup d'une profondeur entièrement terminée est déjà disponible.

5.5 Synthèse

TABLE 3 – Complexité détaillée, méthode par méthode, des classes IA

Classe	Méthode	Complexité	Indice dans le code
SimpleEvaluator	evaluate(Board)	$O(n)$	Une boucle unique parcourt les cases actives de 0 à <code>board.getIndexMax()-1</code> , avec tests de bits en temps constant.
AdvancedEvaluator	evaluate(Board)	$O(n)$	Même schéma que SimpleEvaluator : boucle linéaire + calcul arithmétique constant du bonus d'avancement.
MaxEvaluator	evaluate(Board)	$O(n + M_w + M_b)$	Boucle principale en n , appel de <code>isInCenter(...)</code> en temps constant, puis coût de mobilité via <code>getWhiteValidMoves()</code> et <code>getBlackValidMoves()</code> .
MaxEvaluator	isInCenter(int, int, int)	$O(1)$	Seulement des bornes calculées et des comparaisons.
IterativeDeepening	calculateBestMove(...)	$\sum_{k=1}^d O(b^k) = O(b^d)$	Boucle sur les profondeurs 1 à d , chaque itération appelant MinMax avec profondeur courante.
MinMax	calculateBestMove(...)	$O(b^d)$	Un appel principal à <code>findBestMoveAtDepth(...)</code> à profondeur <code>maxDepth</code> .
MinMax	findBestMoveAtDepth(...)	$O(b^d)$	Boucle sur les coups racines puis descente récursive dans l'arbre de jeu sans élagage.
MinMax	minMax(...)	$O(b^k)$	Pour profondeur restante k , exploration de tous les coups noirs puis appel récursif <code>maxMin(...)</code> .
MinMax	maxMin(...)	$O(b^k)$	Symétrique de <code>minMax(...)</code> pour les coups blancs, sans coupure.
MinMax	checkTime(long)	$O(1)$	Un seul test booléen via <code>isTimeExceeded(startTime)</code> .
MinMaxAlphaBeta	calculateBestMove(...)	Pire : $O(b^d)$	Lance <code>findBestMoveAtDepth(...)</code> ; l'élagage dépend de l'ordre des coups.
MinMaxAlphaBeta	findBestMoveAtDepth(...)	Meilleur : $O(b^{d/2})$ Pire : $O(b^d)$	Mise à jour de α et β à chaque enfant ; coupures quand $\beta \leq \alpha$.
MinMaxAlphaBeta	maxValue(...)	Meilleur : $O(b^{d/2})$ Pire : $O(b^d)$	Exploration récursive avec arrêt anticipé dès qu'une coupure $\beta \leq \alpha$ est détectée.
MinMaxAlphaBeta	minValue(...)	Meilleur : $O(b^{d/2})$ Pire : $O(b^d)$	Symétrique de <code>maxValue(...)</code> avec bornes inversées et mêmes règles de coupure.
MinMaxAlphaBeta	checkTime(long)	$O(1)$	Test constant, puis levée d'exception interne si le temps est dépassé.

6 Explication approfondie

7 Revue critique de projet

7.1 Bug

7.2 Axes d'amélioration

Conclusion

Références

- [1] Brandon Corfman. Raven Checkers — open-source checkers with ai. <https://github.com/bcorfman/raven-checkers>, 2023. Consulté le 6 avril 2026.
- [2] mortexsa. AwsDame — multiplayer checkers platform. <https://github.com/mortexsa/AwsDame>, 2023. Consulté le 6 avril 2026.
- [3] Stuart Russell and Peter Norvig. *Artificial Intelligence : A Modern Approach*. Pearson, 4 edition, 2020.

A Liste des 40 besoins

TABLE 4 – État d’avancement des besoins fonctionnels

ID	Description du besoin	Statut
F1	Lancement du programme	
F2	Fichier de configuration	
F3	Internationalisation (i18n)	
F4	Interface en ligne de commande	
F5	Mode blitz	
F6	Mode contest	
F7	Interface graphique	
F8	Mode IA	
F9	Taille du damier	
F10	Gestion des règles	
F11	Gestion des erreurs de l'utilisateur	
F12	Gestion du undo/redo	
F13	Gestion des conseils	
F14	Gestion du blitz	
F15	Shell interactif	
F16	Proposer de sauvegarder avant de quitter	
F17	Édition de la commande en cours	
F18	Historique des commandes tapées	
F19	Complétion des commandes	
F20	Notation des coups	
F21	Sauvegarde et restauration d'une partie	
F22	Commentaires dans la sauvegarde	
F23	Format des paramètres du jeu	
F24	Format du damier	
F25	Format de l'historique	
F26	Module bitboard	
F27	Algorithmes de manipulation des bitboards	
F28	Menus de l'application	
F29	Raccourcis clavier	
F30	Drag'n'Drop	
F31	Temps de réflexion borné	
F32	Minimax avec $\alpha\beta$ -pruning	
F33	Fonctions d'évaluation pour Minimax	
F34	Approfondissement itératif de Minimax	
F35	Profondeur de recherche de Minimax	
F36	Monte Carlo Tree Search (MCTS)	
F37	Fonction de sélection par Machine Learning	
F38	Découverte de serveurs et connexion client-serveur basique	
F39	Serveur multi-clients et gestion de parties	
F40	Système d'invitations et gestion avancée des joueurs	